

# Paxos (computer science)

---

In computer science, **Paxos** is a family of protocols for solving consensus in a network of unreliable or fallible processors. Consensus is the process of agreeing on one result among a group of participants. This problem becomes difficult when the participants or their communications may experience failures.<sup>[1]</sup>

Consensus protocols are the basis for the state machine replication approach to distributed computing, as suggested by Leslie Lamport<sup>[2]</sup> and surveyed by Fred Schneider.<sup>[3]</sup> State machine replication is a technique for converting an algorithm into a fault-tolerant, distributed implementation. Ad-hoc techniques may leave important cases of failures unresolved. The principled approach proposed by Lamport et al. ensures all cases are handled safely.

The Paxos protocol was first submitted in 1989 and named after a fictional legislative consensus system used on the Paxos island in Greece, where Lamport wrote that the parliament had to function "even though legislators continually wandered in and out of the parliamentary Chamber".<sup>[4]</sup> It was later published as a journal article in 1998.<sup>[5]</sup>

The Paxos family of protocols includes a spectrum of trade-offs between the number of processors, number of message delays before learning the agreed value, the activity level of individual participants, number of messages sent, and types of failures. Although no deterministic fault-tolerant consensus protocol can guarantee progress in an asynchronous network (a result proved in a paper by Fischer, Lynch and Paterson<sup>[6]</sup>), Paxos guarantees safety (consistency), and the conditions that could prevent it from making progress are difficult to provoke.

Paxos is usually used where durability is required (for example, to replicate a file or a database), in which the amount of durable state could be large. The protocol attempts to make progress even during periods when some bounded number of replicas are unresponsive. There is also a mechanism to drop a permanently failed replica or to add a new replica.

## History

---

In 1988, Lynch, Dwork and Stockmeyer had demonstrated the solvability of consensus in a broad family of "partially synchronous" systems.<sup>[7]</sup> Paxos has similarities to a protocol used for agreement in "viewstamped replication", first published by Oki and Liskov in 1988, in the context of distributed transactions.<sup>[8]</sup> Paxos offered an elegant formalism and included one of the earliest proofs of safety for a fault-tolerant distributed consensus protocol.

Reconfigurable state machines have ties to prior work on reliable group multicast protocols that support dynamic group membership e.g. Birman's work in 1985 and 1987 on the virtually synchronous gbcast<sup>[9]</sup> protocol. gbcast is uncommon in supporting durability and addressing partitioning failures. Most reliable multicast protocols do not have these properties which are required for implementations of the state machine replication model. This point is discussed in a paper by Lamport, Malkhi and Zhou.<sup>[10]</sup>

Paxos protocols are members of a theoretical class of solutions to a problem formalized as uniform agreement with crash failures. Lower bounds for this problem have been proved by [Keidar and Shraer](#).<sup>[11]</sup> [Derecho](#),<sup>[12]</sup> a C++ [software library](#) for cloud-scale state machine replication, offers a Paxos protocol that has been integrated with self-managed virtually synchronous membership. This protocol matches the Keidar and Shraer optimality bounds and maps efficiently to modern [remote DMA \(RDMA\)](#) datacenter hardware. It uses TCP if RDMA is not available.

## Assumptions

---

In order to simplify the presentation of Paxos, the following assumptions and definitions are made explicit. Techniques to broaden the applicability are known in the literature, and are not covered in this article.

### Processors

- Processors operate at arbitrary speed.
- Processors may experience failures.
- Processors with stable storage may re-join the protocol after failures (following a crash-recovery failure model).
- Processors do not collude, lie, or otherwise attempt to subvert the protocol. (That is, [Byzantine failures](#) don't occur. See [Byzantine Paxos](#) for a solution that tolerates failures that arise from arbitrary/malicious behavior of the processes.)

### Network

- Processors can send messages to any other processor.
- Messages are sent asynchronously and may take arbitrarily long to deliver.
- Messages may be lost, reordered, or duplicated.
- Messages are delivered without corruption. (That is, [Byzantine failures](#) don't occur. See [Byzantine Paxos](#) for a solution that tolerates corrupted messages that arise from arbitrary/malicious behavior of the messaging channels.)

### Number of processors

In general, a consensus algorithm can make progress using  $n = 2F + 1$  processors, despite the simultaneous failure of any  $F$  processors:<sup>[13]</sup> in other words, the number of non-faulty processes must be strictly greater than the number of faulty processes. However, using reconfiguration, a protocol may be employed which survives any number of total failures as long as no more than  $F$  fail simultaneously. For Paxos protocols, these reconfigurations can be handled as separate *configurations*.<sup>[14]</sup>

## Safety and liveness properties

---

In order to guarantee *safety* (also called "consistency"), Paxos defines three properties and ensures the first two are always held, regardless of the pattern of failures:

**Validity (or *non-triviality*)**

Only proposed values can be chosen and learned.<sup>[15]</sup>

**Agreement (or *consistency*, or *safety*)**

No two distinct learners can learn different values (or there can't be more than one decided value).<sup>[15][16]</sup>

**Termination (or *liveness*)**

If value  $C$  has been proposed, then eventually learner  $L$  will learn some value (if sufficient processors remain non-faulty).<sup>[16]</sup>

Note that Paxos is *not* guaranteed to terminate, and thus does not have the liveness property. This is supported by the Fischer Lynch Paterson impossibility result (FLP)<sup>[6]</sup> which states that a consistency protocol can only have two of *safety*, *liveness*, and *fault tolerance*. As Paxos's point is to ensure fault tolerance and it guarantees safety, it cannot also guarantee liveness.

## Typical deployment

---

In most deployments of Paxos, each participating process acts in three roles: Proposer, Acceptor and Learner.<sup>[17]</sup> This reduces the message complexity significantly, without sacrificing correctness:

In Paxos, clients send commands to a leader. During normal operation, the leader receives a client's command, assigns it a new command number  $i$ , and then begins the  $i^{\text{th}}$  instance of the consensus algorithm by sending messages to a set of acceptor processes.<sup>[16]</sup>

By merging roles, the protocol "collapses" into an efficient client-master-replica style deployment, typical of the database community.<sup>[18]</sup> The benefit of the Paxos protocols (including implementations with merged roles) is the guarantee of its safety properties.

A typical implementation's message flow is covered in the section Multi-Paxos.

## Basic Paxos

---

This protocol is the most basic of the Paxos family. Each "instance" (or "execution") of the basic Paxos protocol decides on a single output value. The protocol proceeds over several rounds. A successful round has 2 phases: phase 1 (which is divided into parts  $a$  and  $b$ ) and phase 2 (also divided into parts  $a$  and  $b$ ). See below the description of the phases. Remember that we assume an asynchronous model, so e.g. a processor may be in one phase while another processor may be in another.

### Phase 1

#### Phase 1a: *Prepare*

A Proposer creates a message, which we call a Prepare. The message is identified with a unique number,  $n$ , which must be greater than any number previously used in a Prepare message by this Proposer. Note that  $n$  is not the value to be proposed; it is simply a

unique identifier of this initial message by the Proposer. In fact, the Prepare message needn't contain the proposed value (often denoted by  $v$ ).

The Proposer chooses at least a Quorum of Acceptors and sends the Prepare message containing  $n$  to them. A Proposer should not initiate Paxos if it cannot communicate with enough Acceptors to constitute a Quorum.

### Phase 1b: *Promise*

The Acceptors wait for a Prepare message from any of the Proposers. When an Acceptor receives a Prepare message, the Acceptor must examine the identifier number,  $n$ , of that message. There are two cases:

1. If  $n$  is higher than every previous proposal number received by the Acceptor (from any Proposer), then the Acceptor must return a message (called a *Promise*) to the Proposer, indicating that the Acceptor will ignore all future proposals numbered less than or equal to  $n$ . The Promise must include the highest number among the Proposals that the Acceptor previously accepted, along with the corresponding accepted value. The first Prepare message vacuously satisfies this condition.
2. If  $n$  is less than or equal to any previous proposal number received by the Acceptor, the Acceptor needn't respond and can ignore the proposal. However, for the sake of optimization, sending a denial, or *negative acknowledgement* (*NAK*), response would tell the Proposer that it can stop its attempt to create consensus with proposal  $n$ .

## Phase 2

### Phase 2a: *Accept*

If a Proposer receives Promises from a Quorum of Acceptors, it needs to set a value  $v$  to its proposal. If any Acceptors had previously accepted any proposal, then they'll have sent their values to the Proposer, who now must set the value of its proposal,  $v$ , to the value associated with the highest proposal number reported by the Acceptors, let's call it  $z$ . If none of the Acceptors had accepted a proposal up to this point, then the Proposer may choose the value it originally wanted to propose, say  $x$ .<sup>[19]</sup>

The Proposer sends an *Accept* message,  $(n, v)$ , to a Quorum of Acceptors with the chosen value for its proposal,  $v$ , and the proposal number  $n$  (which is the same as the number contained in the *Prepare* message previously sent to the Acceptors). So, the *Accept* message is either  $(n, v=z)$  or, in case none of the Acceptors previously accepted a value,  $(n, v=x)$ .

This *Accept* message should be interpreted as a "request", as in "Accept this proposal, please!".

### Phase 2b: *Accepted*

If an Acceptor receives an *Accept* message,  $(n, v)$ , from a Proposer, it must accept it if and only if it has *not* already promised (in Phase 1b of the Paxos protocol) to only consider proposals having an identifier greater than  $n$ .

If the Acceptor has not already promised (in Phase 1b) to only consider proposals having an identifier greater than  $n$ , it should register the value  $v$  (of the just received *Accept* message) as the accepted value (of the Protocol), and send an *Accepted*

message to the Proposer and every Learner (which can typically be the Proposers themselves). Learners will learn the decided value *only after* receiving Accepted messages from a majority of acceptors, i.e. *not* after receiving just the first Accept message.

Else, it can ignore the Accept message or request.

Note that consensus is achieved when a majority of Acceptors accept the same *identifier number* (rather than the same *value*). Because each identifier is unique to a Proposer and only one value may be proposed per identifier, all Acceptors that accept the same identifier thereby accept the same value. These facts result in a few counter-intuitive scenarios that do not impact correctness: Acceptors can accept multiple values, a value may achieve a majority across Acceptors (with different identifiers) only to later be changed, and Acceptors may continue to accept proposals after an identifier has achieved a majority. However, the Paxos protocol guarantees that consensus is permanent and the chosen value is immutable.

## When rounds fail

Rounds fail when multiple Proposers send conflicting *Prepare* messages, or when the Proposer does not receive a Quorum of responses (*Promise* or *Accepted*). In these cases, another round must be started with a higher proposal number.

## Paxos can be used to select a leader

Notice that a Proposer in Paxos could propose "I am the leader" (or, for example, "Proposer X is the leader").<sup>[20]</sup> Because of the agreement and validity guarantees of Paxos, if accepted by a Quorum, then the Proposer is now known to be the leader to all other nodes. This satisfies the needs of leader election<sup>[21]</sup> because there is a single node believing it is the leader and a single node known to be the leader at all times.

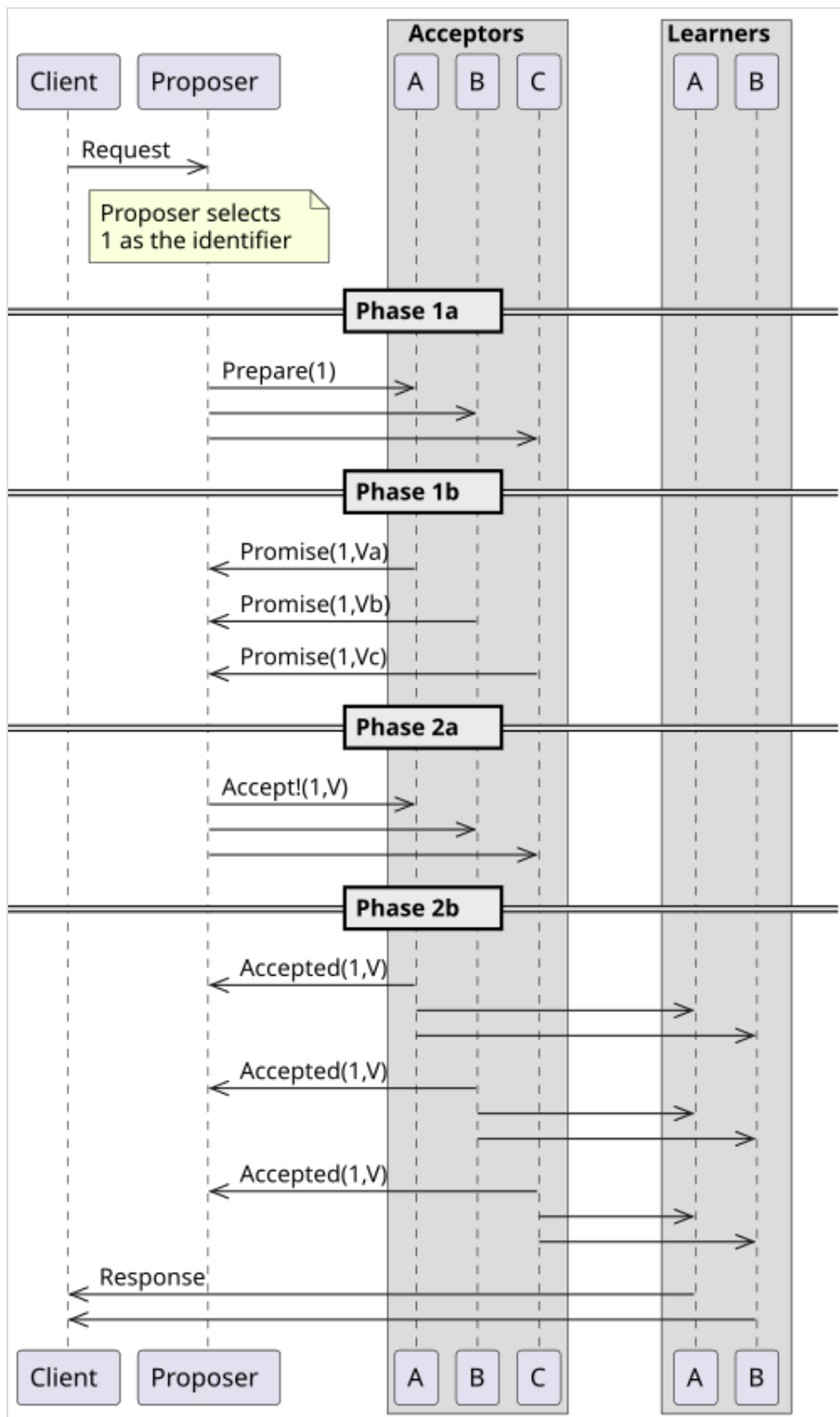
## Graphic representation of the flow of messages in the basic Paxos

The following diagrams represent several cases/situations of the application of the Basic Paxos protocol. Some cases show how the Basic Paxos protocol copes with the failure of certain (redundant) components of the distributed system.

Note that the values returned in the *Promise* message are "null" the first time a proposal is made (since no Acceptor has accepted a value before in this round).

### Basic Paxos without failures

In the diagram below, there is 1 Client, 1 Proposer, 3 Acceptors (i.e. the Quorum size is 3) and 2 Learners (represented by the 2 vertical lines). This diagram represents the case of a first round, which is successful (i.e. no process in the network fails).



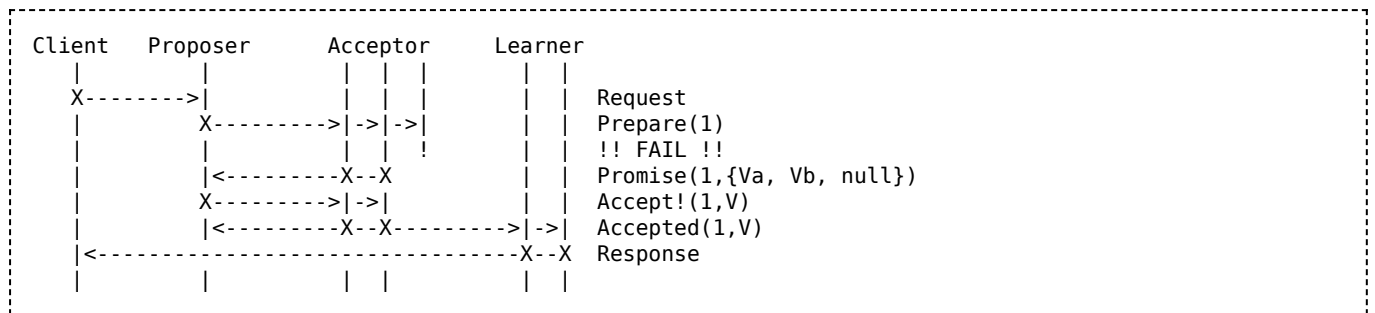
Here, V is the last of (Va, Vb, Vc).

### Error cases in basic Paxos

The simplest error cases are the failure of an Acceptor (when a Quorum of Acceptors remains alive) and failure of a redundant Learner. In these cases, the protocol requires no "recovery" (i.e. it still succeeds): no additional rounds or messages are required, as shown below (in the next two diagrams/cases).

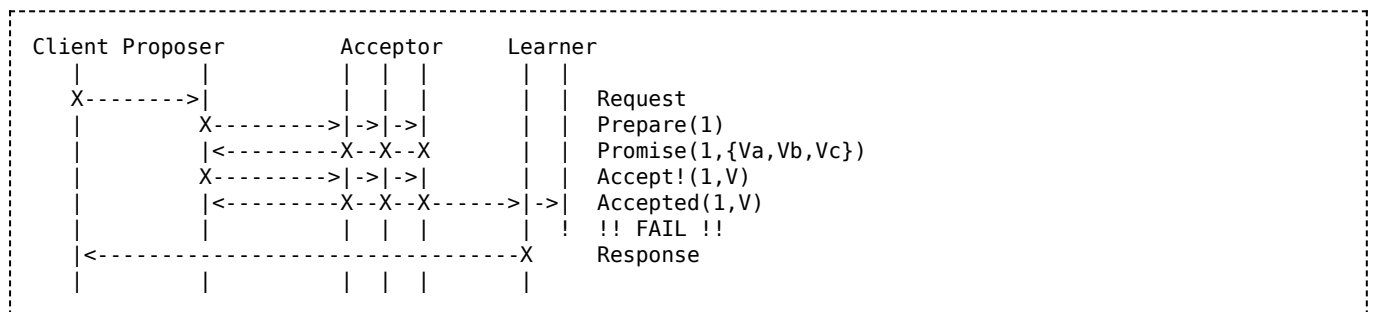
## Basic Paxos when an Acceptor fails

In the following diagram, one of the Acceptors in the Quorum fails, so the Quorum size becomes 2. In this case, the Basic Paxos protocol still succeeds.



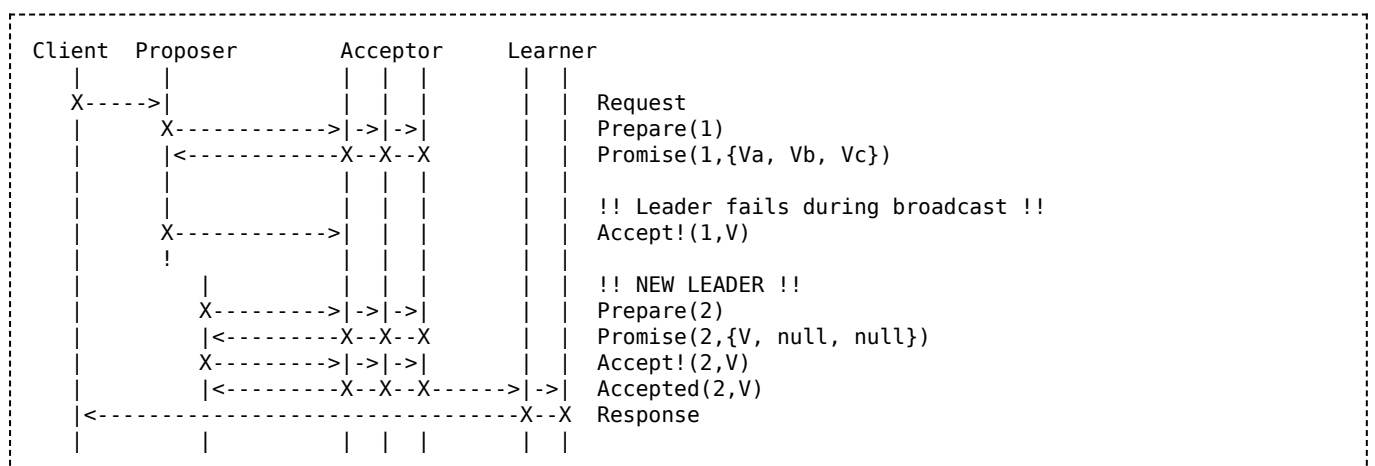
## Basic Paxos when a redundant learner fails

In the following case, one of the (redundant) Learners fails, but the Basic Paxos protocol still succeeds.



## Basic Paxos when a Proposer fails

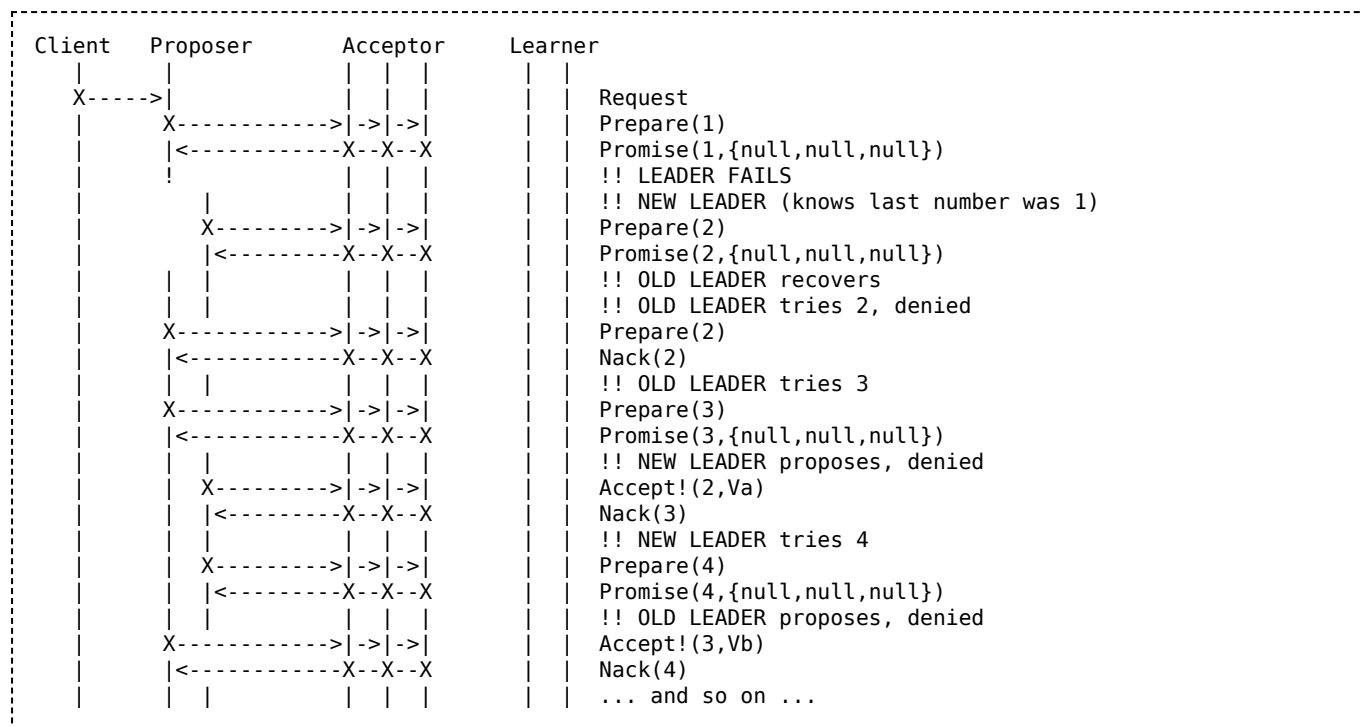
In this case, a Proposer fails after proposing a value, but before the agreement is reached. Specifically, it fails in the middle of the Accept message, so only one Acceptor of the Quorum receives the value. Meanwhile, a new Leader (a Proposer) is elected (but this is not shown in detail). Note that there are 2 rounds in this case (rounds proceed vertically, from the top to the bottom).



## Basic Paxos when multiple Proposers conflict

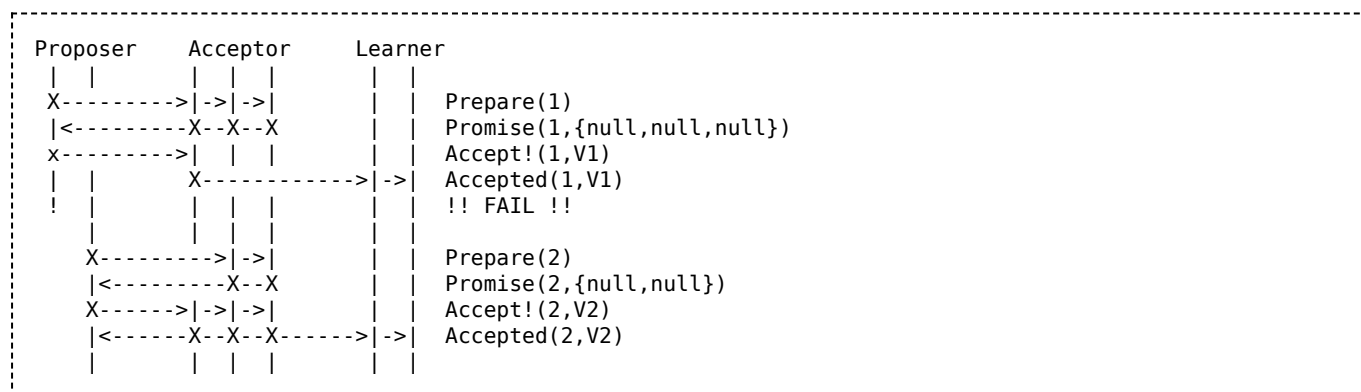
The most complex case is when multiple Proposers believe themselves to be Leaders. For instance, the current leader may fail and later recover, but the other Proposers have already re-selected a new leader. The recovered leader has not learned this yet and attempts to begin one round in conflict with the current

leader. In the diagram below, 4 unsuccessful rounds are shown, but there could be more (as suggested at the bottom of the diagram).



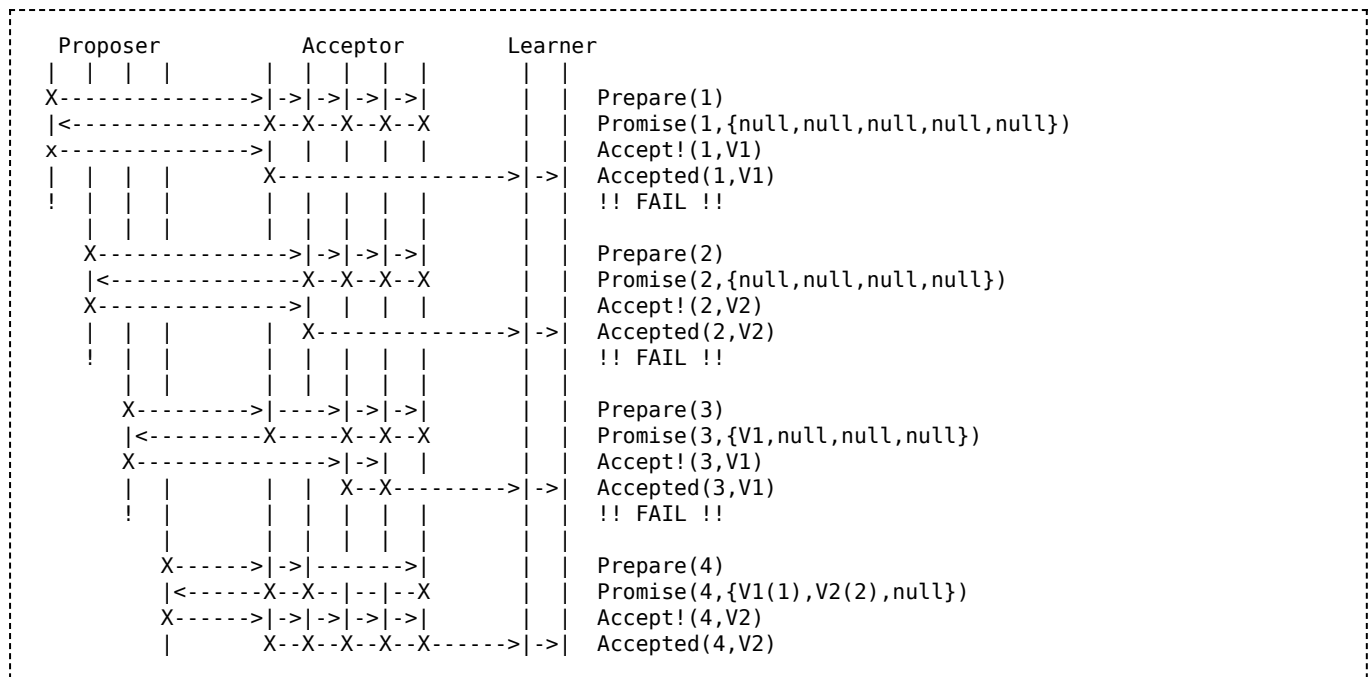
## Basic Paxos where an Acceptor accepts Two Different Values

In the following case, one Proposer achieves acceptance of value V1 by one Acceptor before failing. A new Proposer prepares the Acceptors that never accepted V1, allowing it to propose V2. Then V2 is accepted by all Acceptors, including the one that initially accepted V1.



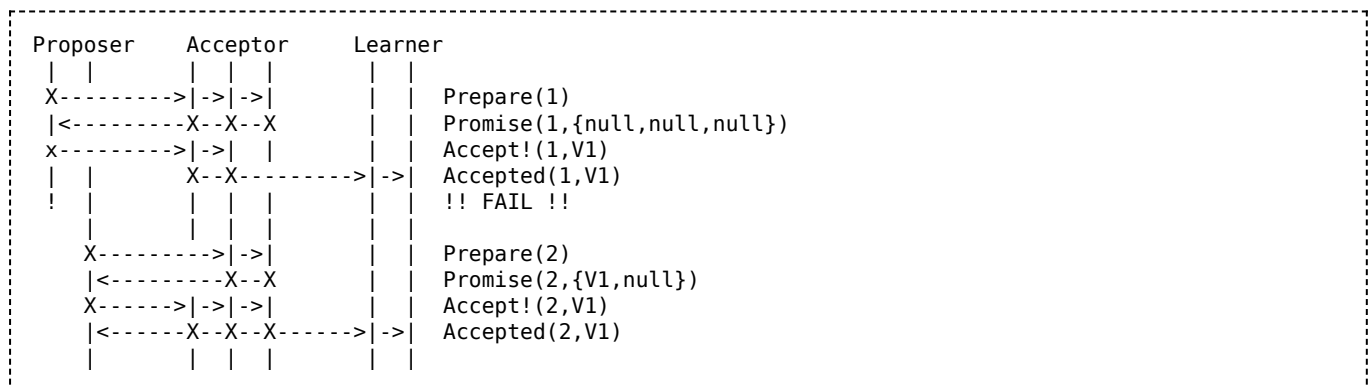
## Basic Paxos where a multi-identifier majority is insufficient

In the following case, one Proposer achieves acceptance of value V1 of one Acceptor before failing. A new Proposer prepares the Acceptors that never accepted V1, allowing it to propose V2. This Proposer is able to get one Acceptor to accept V2 before failing. A new Proposer finds a majority that includes the Acceptor that has accepted V1, and must propose it. The Proposer manages to get two Acceptors to accept it before failing. At this point, three Acceptors have accepted V1, but not for the same identifier. Finally, a new Proposer prepares the majority that has not seen the largest accepted identifier. The value associated with the largest identifier in that majority is V2, so it must propose it. This Proposer then gets all Acceptors to accept V2, achieving consensus.



## Basic Paxos where new Proposers cannot change an existing consensus

In the following case, one Proposer achieves acceptance of value V1 of two Acceptors before failing. A new Proposer may start another round, but it is now impossible for that proposer to prepare a majority that doesn't include at least one Acceptor that has accepted V1. As such, even though the Proposer doesn't see the existing consensus, the Proposer's only option is to propose the value already agreed upon. New Proposers can continually increase the identifier to restart the process, but the consensus can never be changed.



## Multi-Paxos

A typical deployment of Paxos requires a continuous stream of agreed values acting as commands to a distributed state machine. If each command is the result of a single instance of the Basic Paxos protocol, a significant amount of overhead would result.

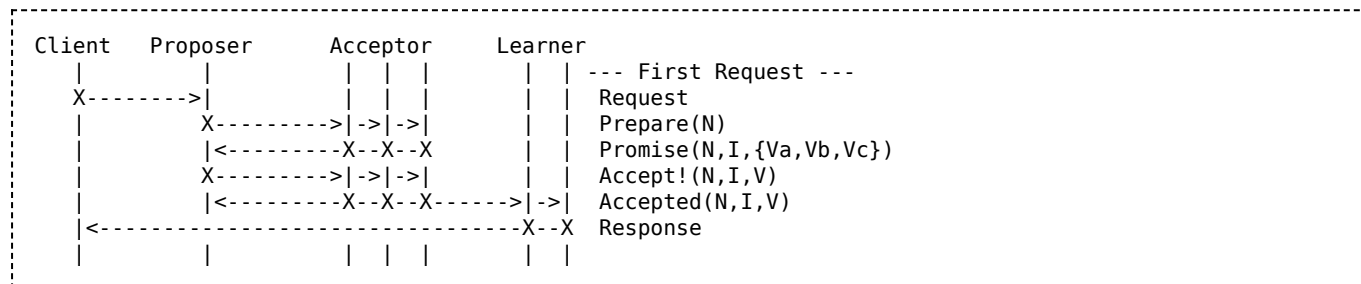
If the leader is relatively stable, phase 1 becomes unnecessary. Thus, it is possible to skip phase 1 for future instances of the protocol with the same leader.

To achieve this, the round number  $I$  is included along with each value which is incremented in each round by the same Leader. Multi-Paxos reduces the failure-free message delay (proposal to learning) from 4 delays to 2 delays.

## Graphic representation of the flow of messages in the Multi-Paxos

### Multi-Paxos without failures

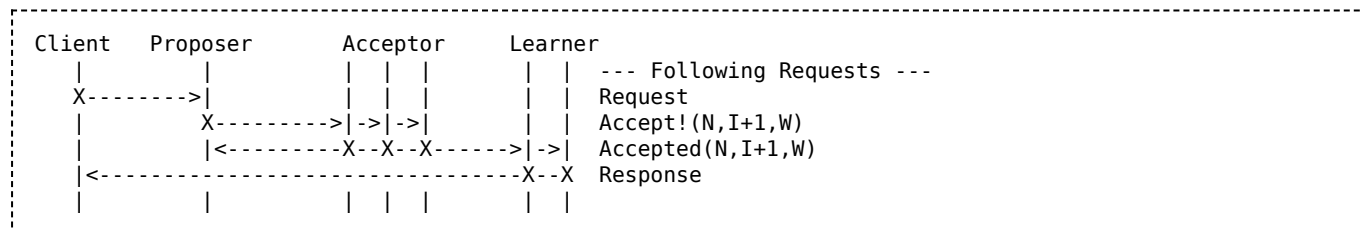
In the following diagram, only one instance (or "execution") of the basic Paxos protocol, with an initial Leader (a Proposer), is shown. Note that a Multi-Paxos consists of several instances of the basic Paxos protocol.



where  $V = \text{last of } (V_a, V_b, V_c)$ .

### Multi-Paxos when phase 1 can be skipped

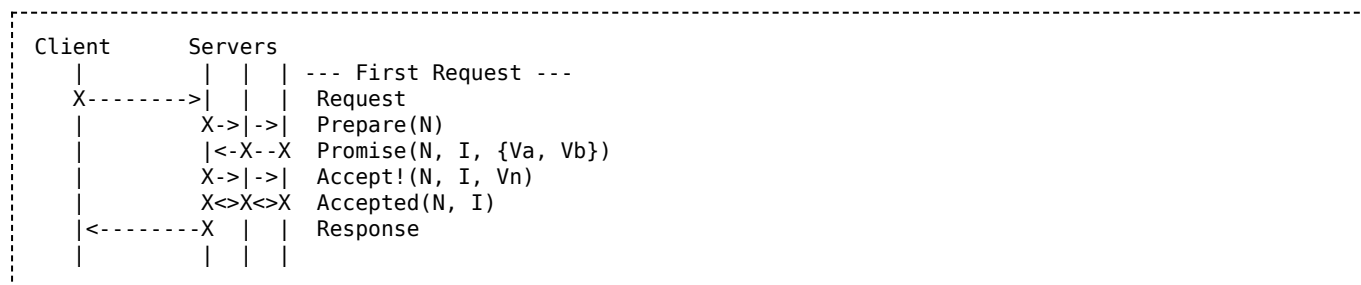
In this case, subsequent instances of the basic Paxos protocol (represented by  $I+1$ ) use the same leader, so the phase 1 (of these subsequent instances of the basic Paxos protocol), which consist of the Prepare and Promise sub-phases, is skipped. Note that the Leader should be stable, i.e. it should not crash or change.



### Multi-Paxos when roles are collapsed

A common deployment of the Multi-Paxos consists in collapsing the role of the Proposers, Acceptors and Learners to "Servers". So, in the end, there are only "Clients" and "Servers".

The following diagram represents the first "instance" of a basic Paxos protocol, when the roles of the Proposer, Acceptor and Learner are collapsed to a single role, called the "Server".



## Multi-Paxos when roles are collapsed and the leader is steady

In the subsequent instances of the basic Paxos protocol, with the same leader as in the previous instances of the basic Paxos protocol, the phase 1 can be skipped.



## Optimisations

A number of optimisations can be performed to reduce the number of exchanged messages, to improve the performance of the protocol, etc. A few of these optimisations are reported below.

"We can save messages at the cost of an extra message delay by having a single distinguished learner that informs the other learners when it finds out that a value has been chosen. Acceptors then send *Accepted* messages only to the distinguished learner. In most applications, the roles of leader and distinguished learner are performed by the same processor.<sup>[22]</sup>

"A leader can send its *Prepare* and *Accept!* messages just to a quorum of acceptors. As long as all acceptors in that quorum are working and can communicate with the leader and the learners, there is no need for acceptors not in the quorum to do anything.<sup>[22]</sup>

"Acceptors do not care what value is chosen. They simply respond to *Prepare* and *Accept!* messages to ensure that, despite failures, only a single value can be chosen. However, if an acceptor does learn what value has been chosen, it can store the value in stable storage and erase any other information it has saved there. If the acceptor later receives a *Prepare* or *Accept!* message, instead of performing its Phase1b or Phase2b action, it can simply inform the leader of the chosen value.<sup>[22]</sup>

"Instead of sending the value *v*, the leader can send a hash of *v* to some acceptors in its *Accept!* messages. A learner will learn that *v* is chosen if it receives *Accepted* messages for either *v* or its hash from a quorum of acceptors, and at least one of those messages contains *v* rather than its hash. However, a leader could receive *Promise* messages that tell it the hash of a value *v* that it must use in its Phase2a action without telling it the actual value of *v*. If that happens, the leader cannot execute its Phase2a action until it communicates with some process that knows *v*.<sup>[22]</sup>

"A proposer can send its proposal only to the leader rather than to all coordinators. However, this requires that the result of the leader-selection algorithm be broadcast to the proposers, which might be expensive. So, it might be better to let the proposer send its proposal to all coordinators. (In that case, only the coordinators themselves need to know who the leader is.)<sup>[15]</sup>

"Instead of each acceptor sending *Accepted* messages to each learner, acceptors can send their *Accepted* messages to the leader and the leader can inform the learners when a value has been chosen. However, this adds an extra message delay.<sup>[15]</sup>

"Finally, observe that phase 1 is unnecessary for round 1 .. The leader of round 1 can begin the round by sending an *Accept!* message with any proposed value.<sup>[15]</sup>

## Cheap Paxos

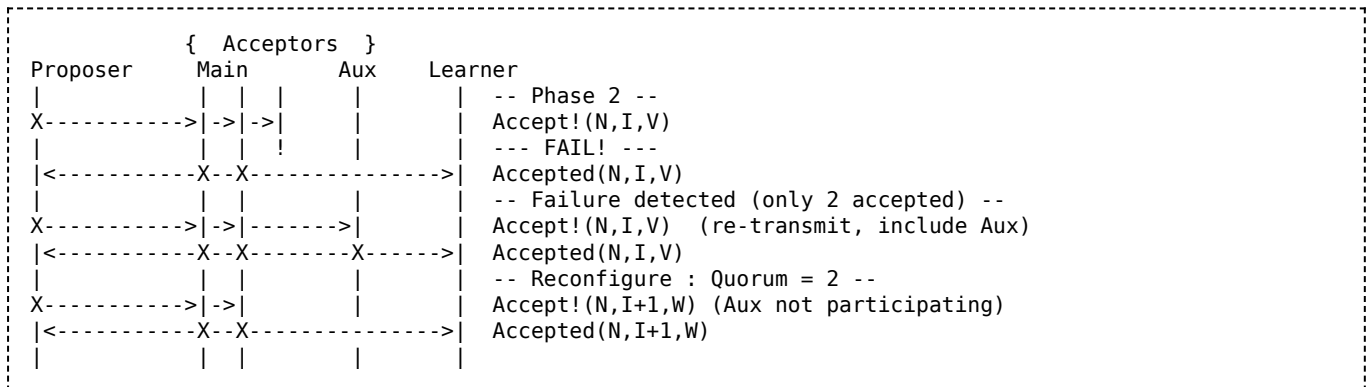
Cheap Paxos extends Basic Paxos to tolerate  $F$  failures with  $F+1$  main processors and  $F$  auxiliary processors by dynamically reconfiguring after each failure.

This reduction in processor requirements comes at the expense of liveness; if too many main processors fail in a short time, the system must halt until the auxiliary processors can reconfigure the system. During stable periods, the auxiliary processors take no part in the protocol.

"With only two processors  $p$  and  $q$ , one processor cannot distinguish failure of the other processor from failure of the communication medium. A third processor is needed. However, that third processor does not have to participate in choosing the sequence of commands. It must take action only in case  $p$  or  $q$  fails, after which it does nothing while either  $p$  or  $q$  continues to operate the system by itself. The third processor can therefore be a small/slow/cheap one, or a processor primarily devoted to other tasks."<sup>[22]</sup>

### Message flow: Cheap Multi-Paxos

An example involving three main acceptors, one auxiliary acceptor and quorum size of three, showing failure of one main processor and subsequent reconfiguration:



## Fast Paxos

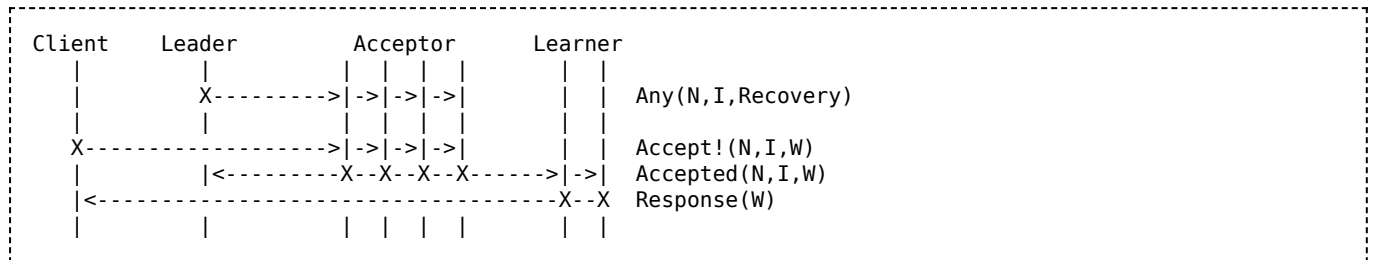
Fast Paxos generalizes Basic Paxos to reduce end-to-end message delays. In Basic Paxos, the message delay from client request to learning is 3 message delays. Fast Paxos allows 2 message delays, but requires that (1) the system be composed of  $3f+1$  acceptors to tolerate up to  $f$  faults (instead of the classic  $2f+1$ ), and (2) the Client to send its request to multiple destinations.

Intuitively, if the leader has no value to propose, then a client could send an *Accept!* message to the Acceptors directly. The Acceptors would respond as in Basic Paxos, sending *Accepted* messages to the leader and every Learner achieving two message delays from Client to Learner.

If the leader detects a collision, it resolves the collision by sending *Accept!* messages for a new round which are *Accepted* as usual. This coordinated recovery technique requires four message delays from Client to Learner.

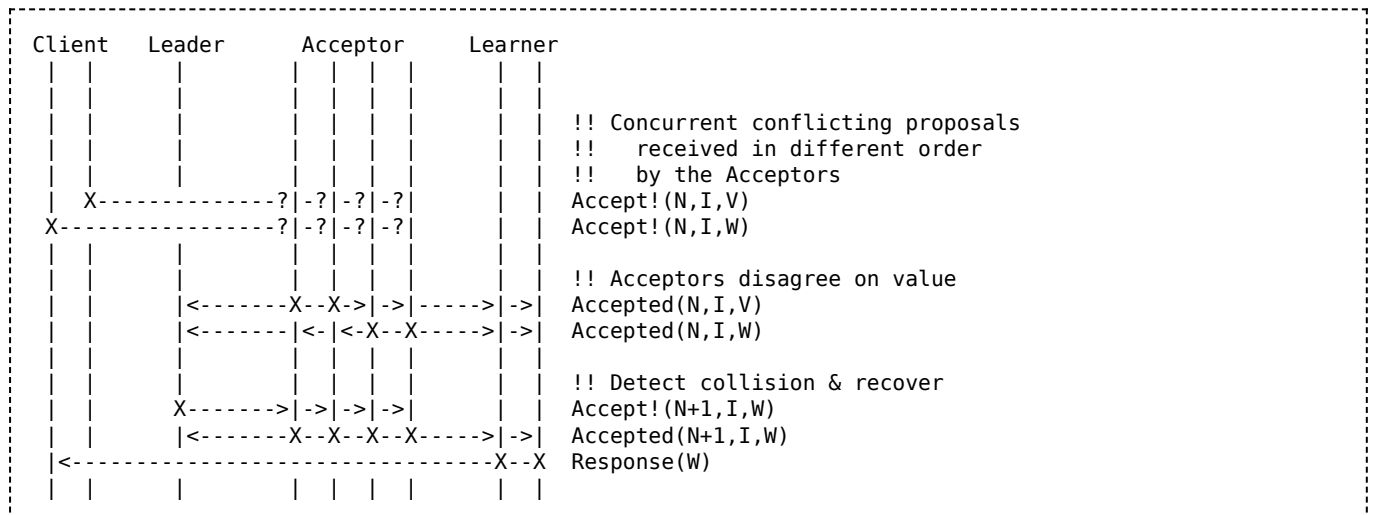
The final optimization occurs when the leader specifies a recovery technique in advance, allowing the Acceptors to perform the collision recovery themselves. Thus, uncoordinated collision recovery can occur in three message delays (and only two message delays if all Learners are also Acceptors).

## Message flow: Fast Paxos, non-conflicting

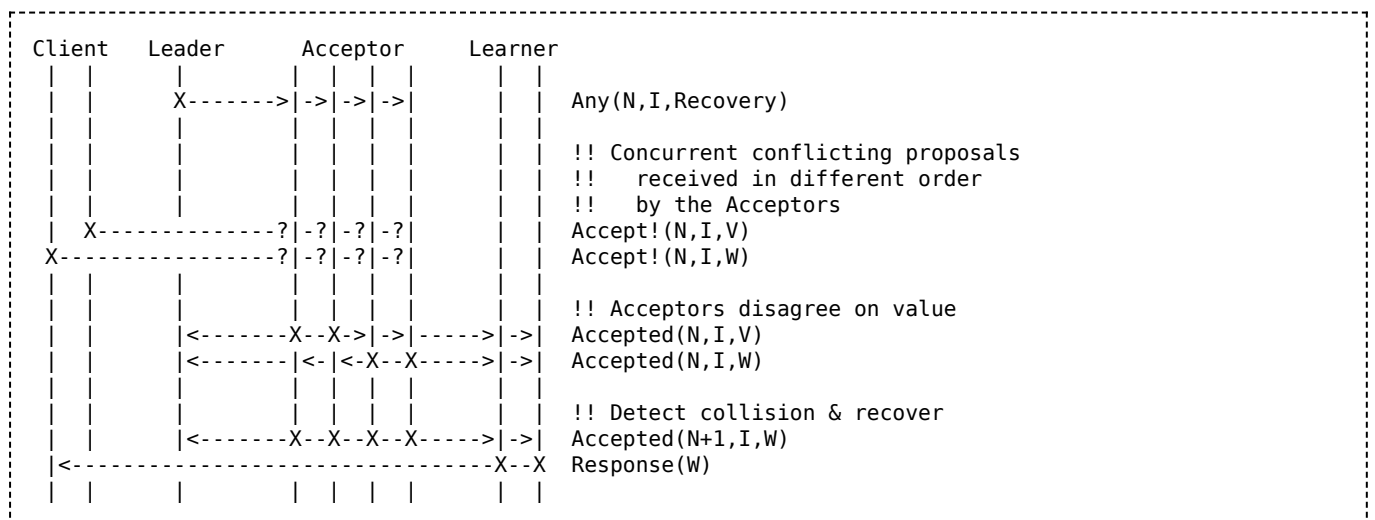


## Message flow: Fast Paxos, conflicting proposals

Conflicting proposals with coordinated recovery. Note: the protocol does not specify how to handle the dropped client request.

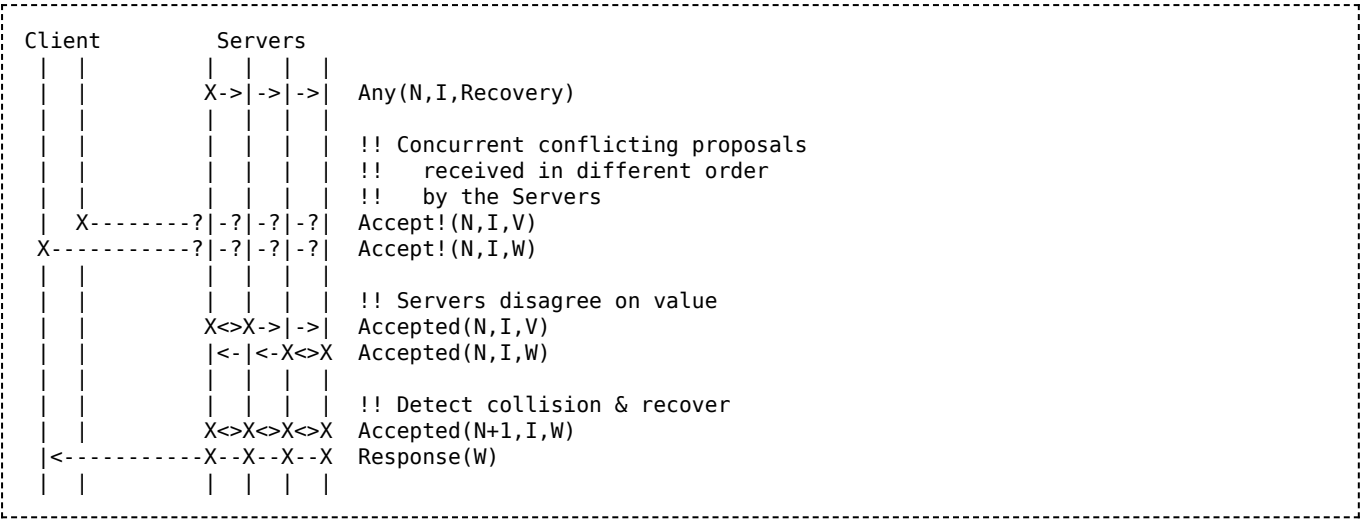


Conflicting proposals with uncoordinated recovery.



# Message flow: Fast Paxos with uncoordinated recovery, collapsed roles

(merged Acceptor/Learner roles)



## Generalized Paxos

Generalized consensus explores the relationship between the operations of the replicated state machine and the consensus protocol that implements it.<sup>[16]</sup> The main discovery involves optimizations of Paxos when conflicting proposals could be applied in any order. i.e., when the proposed operations are commutative operations for the state machine. In such cases, the conflicting operations can both be accepted, avoiding the delays required for resolving conflicts and re-proposing the rejected operations.

This concept is further generalized into ever-growing sequences of commutative operations, some of which are known to be stable (and thus may be executed). The protocol tracks these sequences ensuring that all proposed operations of one sequence are stabilized before allowing any operation non-commuting with them to become stable.

### Example

In order to illustrate Generalized Paxos, the example below shows a message flow between two concurrently executing clients and a replicated state machine implementing read/write operations over two distinct registers A and B.

| Commutativity Table |         |          |         |          |
|---------------------|---------|----------|---------|----------|
|                     | Read(A) | Write(A) | Read(B) | Write(B) |
| Read(A)             |         | ✗        |         |          |
| Write(A)            | ✗       | ✗        |         |          |
| Read(B)             |         |          |         | ✗        |
| Write(B)            |         |          | ✗       | ✗        |

Note that ✗ in this table indicates operations which are non-commutative.

A possible sequence of operations :

<1:Read(A), 2:Read(B), 3:Write(B), 4:Read(B), 5:Read(A), 6:Write(A)>

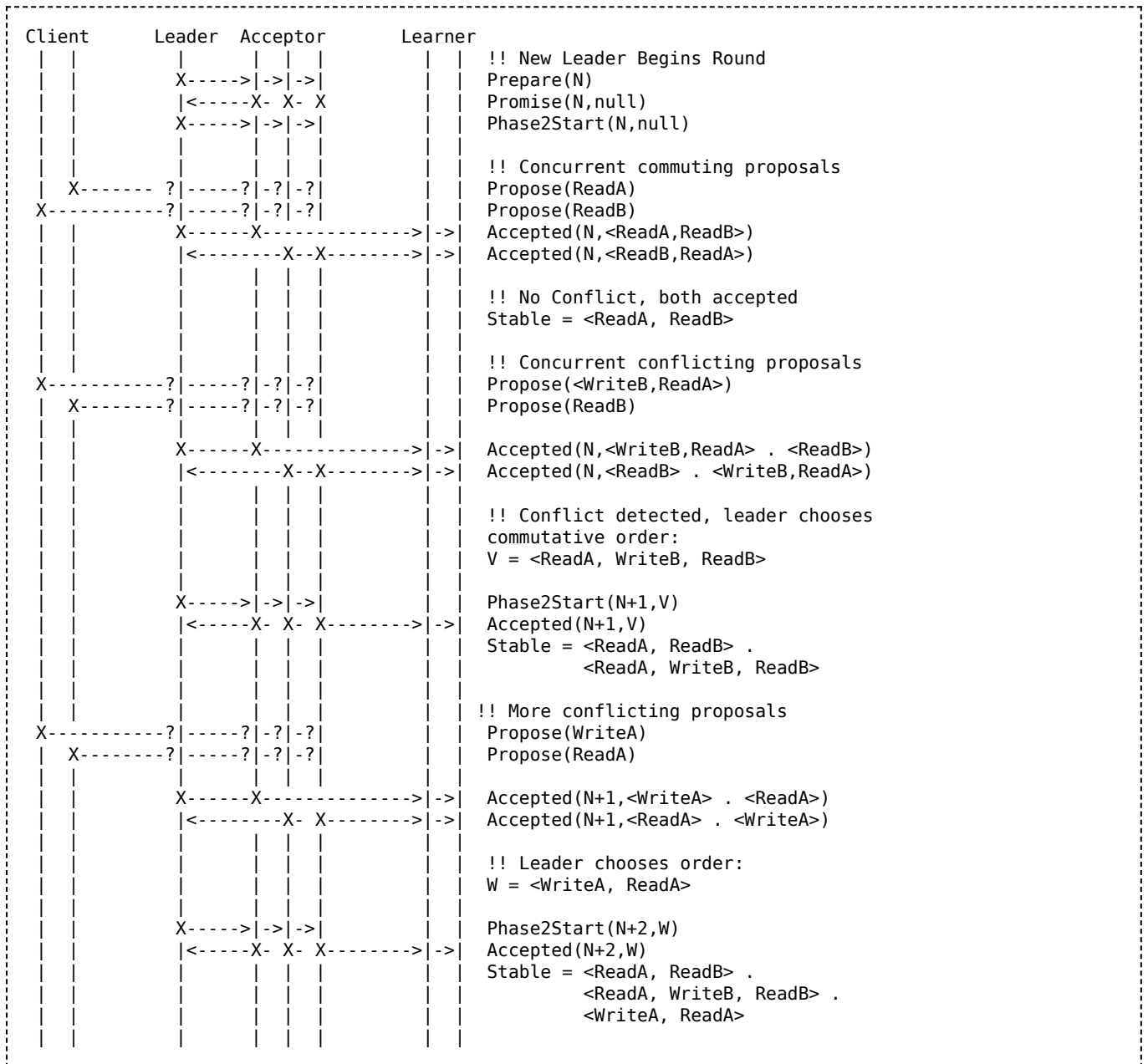
Since 5:Read(A) commutes with both 3:Write(B) and 4:Read(B), one possible permutation equivalent to the previous order is the following:

<1:Read(A), 2:Read(B), 5:Read(A), 3:Write(B), 4:Read(B), 6:Write(A)>

In practice, a commute occurs only when operations are proposed concurrently.

## Message flow: Generalized Paxos (example)

Responses not shown. Note: message abbreviations differ from previous message flows due to specifics of the protocol, see [\[23\]](#) for a full discussion.



## Performance

The above message flow shows us that Generalized Paxos can leverage operation semantics to avoid collisions when the spontaneous ordering of the network fails. This allows the protocol to be in practice quicker than Fast Paxos. However, when a collision occurs, Generalized Paxos needs two additional round trips to recover. This situation is illustrated with operations WriteB and ReadB in the above schema.

In the general case, such round trips are unavoidable and come from the fact that multiple commands can be accepted during a round. This makes the protocol more expensive than Paxos when conflicts are frequent. Hopefully two possible refinements of Generalized Paxos are possible to improve recovery time.<sup>[24]</sup>

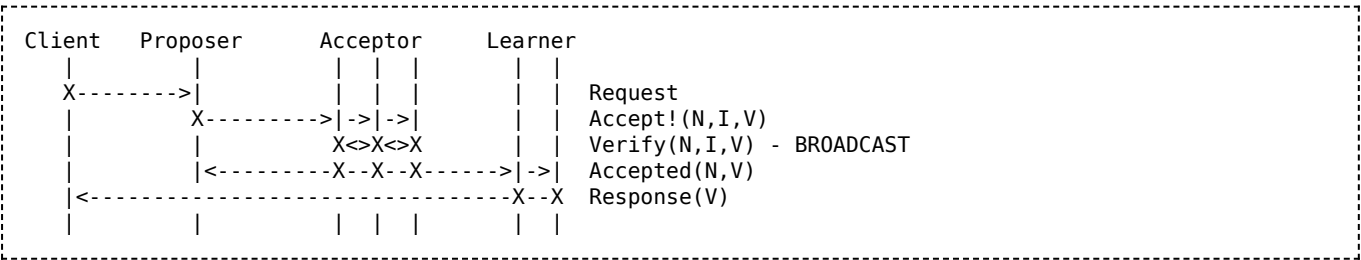
- First, if the coordinator is part of every quorum of acceptors (round N is said *centered*), then to recover at round N+1 from a collision at round N, the coordinator skips phase 1 and proposes at phase 2 the sequence it accepted last during round N. This reduces the cost of recovery to a single round trip.
- Second, if both rounds N and N+1 use a unique and identical centered quorum, when an acceptor detects a collision at round N, it spontaneously proposes at round N+1 a sequence suffixing both (i) the sequence accepted at round N by the coordinator and (ii) the greatest non-conflicting prefix it accepted at round N. For instance, if the coordinator and the acceptor accepted respectively at round N <WriteB, ReadB> and <ReadB, ReadA> , the acceptor will spontaneously accept <WriteB, ReadB, ReadA> at round N+1. With this variation, the cost of recovery is a single message delay which is obviously optimal. Notice here that the use of a unique quorum at a round does not harm liveness. This comes from the fact that any process in this quorum is a read quorum for the prepare phase of the next rounds.<sup>[25]</sup>

## Byzantine Paxos

Paxos may also be extended to support arbitrary failures of the participants, including lying, fabrication of messages, collusion with other participants, selective non-participation, etc. These types of failures are called Byzantine failures, after the solution popularized by Lamport.<sup>[26]</sup>

Byzantine Paxos<sup>[27]</sup> introduced by Castro and Liskov adds an extra message (Verify) which acts to distribute knowledge and verify the actions of the other processors:

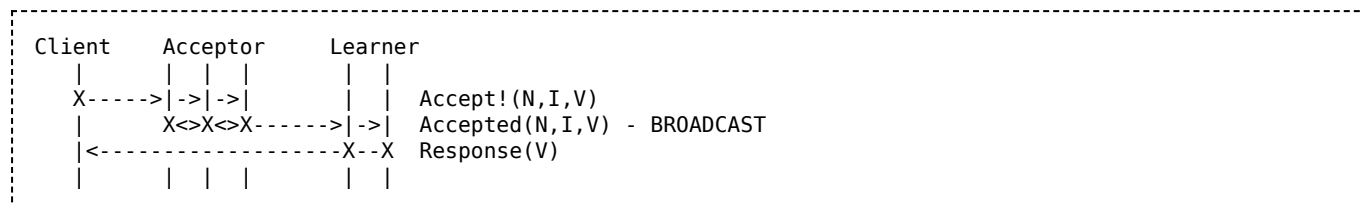
### Message flow: Byzantine Multi-Paxos, steady state



Fast Byzantine Paxos<sup>[28]</sup> introduced by Martin and Alvisi removes this extra delay, since the client sends commands directly to the Acceptors.

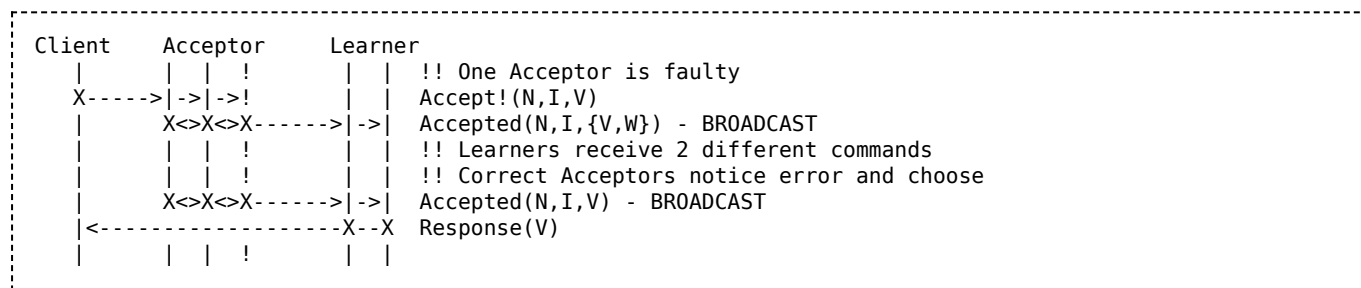
Note the *Accepted* message in Fast Byzantine Paxos is sent to all Acceptors and all Learners, while Fast Paxos sends *Accepted* messages only to Learners):

## Message flow: Fast Byzantine Multi-Paxos, steady state



The failure scenario is the same for both protocols; Each Learner waits to receive  $F+1$  identical messages from different Acceptors. If this does not occur, the Acceptors themselves will also be aware of it (since they exchanged each other's messages in the broadcast round), and correct Acceptors will re-broadcast the agreed value:

## Message flow: Fast Byzantine Multi-Paxos, failure



## Adapting Paxos for RDMA networks

With the emergence of very high speed reliable datacenter networks that support remote DMA (RDMA), there has been substantial interest in optimizing Paxos to leverage hardware offloading, in which the network interface card and network routers provide reliability and network-layer congestion control, freeing the host CPU for other tasks. The Derecho C++ Paxos library is an open-source Paxos implementation that explores this option.<sup>[12]</sup>

Derecho offers both a classic Paxos, with data durability across full shutdown/restart sequences, and vertical Paxos (atomic multicast), for in-memory replication and state-machine synchronization. The Paxos protocols employed by Derecho needed to be adapted to maximize asynchronous data streaming and remove other sources of delay on the leader's critical path. So doing enables Derecho to sustain the full bidirectional RDMA data rate. In contrast, although traditional Paxos protocols can be migrated to an RDMA network by simply mapping the message send operations to native RDMA operations, doing so leaves round-trip delays on the critical path. In high-speed RDMA networks, even small delays can be large enough to prevent utilization of the full potential bandwidth.

## Production use of Paxos

- Google uses the Paxos algorithm in their Chubby distributed lock service in order to keep replicas consistent in case of failure.<sup>[29]</sup> Chubby is used by Bigtable which is now in production in Google Analytics and other products.
- Google Spanner and Megastore use the Paxos algorithm internally.
- The OpenReplica replication service uses Paxos to maintain replicas for an open access system that enables users to create fault-tolerant objects. It provides high performance through concurrent rounds and flexibility through dynamic membership changes.
- IBM supposedly uses the Paxos algorithm in their IBM SAN Volume Controller product to implement a general purpose fault-tolerant virtual machine used to run the configuration and control components of the storage virtualization services offered by the cluster.<sup>[30]</sup>
- Microsoft uses Paxos in the Autopilot cluster management service from Bing, and in Windows Server Failover Clustering.<sup>[31]</sup>
- WANdisco have implemented Paxos within their DConE active-active replication technology.<sup>[32]</sup>
- XtreemFS uses a Paxos-based lease negotiation algorithm for fault-tolerant and consistent replication of file data and metadata.<sup>[33]</sup>
- Heroku uses Doozerd which implements Paxos for its consistent distributed data store.
- Ceph uses Paxos as part of the monitor processes to agree which OSDs are up and in the cluster.
- The MariaDB Xpand distributed SQL database uses Paxos for distributed transaction resolution.<sup>[34]</sup>
- Neo4j HA graph database implements Paxos, replacing Apache ZooKeeper from v1.9
- Apache Cassandra NoSQL database uses Paxos for Light Weight Transaction feature only.<sup>[35]</sup>
- ScyllaDB NoSQL database uses Paxos for Light Weight Transactions.<sup>[36]</sup>
- Amazon Elastic Container Services uses Paxos to maintain a consistent view of cluster state.<sup>[37]</sup>
- Amazon DynamoDB uses the Paxos algorithm for leader election and consensus.<sup>[38]</sup>

## See also

---

- Two generals problem
- Chandra–Toueg consensus algorithm
- State machine
- Raft

## References

---

1. Pease, Marshall; Shostak, Robert; Lamport, Leslie (April 1980). "Reaching Agreement in the Presence of Faults" (<http://research.microsoft.com/users/lamport/pubs/pubs.html#reaching>). *Journal of the Association for Computing Machinery*. **27** (2): 228–234. doi:10.1145/322186.322188 (<https://doi.org/10.1145/322186.322188>). S2CID 6429068 (<https://api.semanticscholar.org/CorpusID:6429068>). Retrieved 2007-02-02.

2. Lamport, Leslie (July 1978). "Time, Clocks and the Ordering of Events in a Distributed System" (<http://research.microsoft.com/users/lamport/pubs/pubs.html#time-clocks>). *Communications of the ACM*. **21** (7): 558–565. doi:10.1145/359545.359563 (<https://doi.org/10.1145%2F359545.359563>). S2CID 215822405 (<https://api.semanticscholar.org/CorpusID:215822405>). Retrieved 2007-02-02.
3. Schneider, Fred (1990). "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial" (<https://www.cs.cornell.edu/fbs/publications/smsurvey.pdf>) (PDF). *ACM Computing Surveys*. **22** (4): 299–319. doi:10.1145/98163.98167 (<https://doi.org/10.1145%2F98163.98167>). S2CID 678818 (<https://api.semanticscholar.org/CorpusID:678818>).
4. Leslie Lamport's history of the paper (<http://research.microsoft.com/en-us/um/people/lamport/pubs/pubs.html#lamport-paxos>)
5. Lamport, Leslie (May 1998). "The Part-Time Parliament" (<http://research.microsoft.com/user/s/lamport/pubs/pubs.html#lamport-paxos>). *ACM Transactions on Computer Systems*. **16** (2): 133–169. doi:10.1145/279227.279229 (<https://doi.org/10.1145%2F279227.279229>). S2CID 421028 (<https://api.semanticscholar.org/CorpusID:421028>). Retrieved 2007-02-02.
6. Fischer, M. (April 1985). "Impossibility of distributed consensus with one faulty process" (<https://doi.org/10.1145%2F3149.214121>). *Journal of the ACM*. **32** (2): 374–382. doi:10.1145/3149.214121 (<https://doi.org/10.1145%2F3149.214121>). hdl:1721.1/149560 (<https://hdl.handle.net/1721.1%2F149560>). S2CID 207660233 (<https://api.semanticscholar.org/CorpusID:207660233>).
7. Dwork, Cynthia; Lynch, Nancy; Stockmeyer, Larry (April 1988). "Consensus in the Presence of Partial Synchrony" (<http://groups.csail.mit.edu/tds/papers/Lynch/jacm88.pdf>) (PDF). *Journal of the ACM*. **35** (2): 288–323. doi:10.1145/42282.42283 (<https://doi.org/10.1145%2F42282.42283>). S2CID 17007235 (<https://api.semanticscholar.org/CorpusID:17007235>).
8. Oki, Brian; Liskov, Barbara (1988). "Viewstamped Replication: A New Primary Copy Method to Support Highly-Available Distributed Systems" (<http://portal.acm.org/citation.cfm?id=62549>). *PODC '88: Proceedings of the seventh annual ACM Symposium on Principles of Distributed Computing*. pp. 8–17. doi:10.1145/62546.62549 (<https://doi.org/10.1145%2F62546.62549>).
9. Birman, Kenneth; Joseph, Thomas (February 1987). "Reliable Communication in the Presence of Failures". *ACM Transactions on Computer Systems*. **5**: 47–76. doi:10.1145/7351.7478 (<https://doi.org/10.1145%2F7351.7478>). hdl:1813/6534 (<https://hdl.handle.net/1813%2F6534>). S2CID 11224827 (<https://api.semanticscholar.org/CorpusID:11224827>).
10. Lamport, Leslie; Malkhi, Dahlia; Zhou, Lidong (March 2010). "Reconfiguring a State Machine". *SIGACT News*. **41** (1): 63–73. doi:10.1145/1753171.1753191 (<https://doi.org/10.1145%2F1753171.1753191>). S2CID 15189602 (<https://api.semanticscholar.org/CorpusID:15189602>).
11. Keidar, Idit; Shraer, Alexander (2006). "Timeliness, failure-detectors, and consensus performance.". *PODC '06: Proceedings of the 25th Annual ACM Symposium on Principles of Distributed Computing*. doi:10.1145/1146381.1146408 (<https://doi.org/10.1145%2F1146381.1146408>).
12. Jha, Sagar; Behrens, Jonathan; Gkountouvas, Theo; Milano, Matthew; Song, Weijia; Tremel, Edward; van Renesse, Robbert; Zink, Sydney; Birman, Ken (April 2019). "Derecho: Fast State Machine Replication for Cloud Services". *ACM Transactions on Computer Systems*. **36** (2). doi:10.1145/3302258 (<https://doi.org/10.1145%2F3302258>). S2CID 218482757 (<https://api.semanticscholar.org/CorpusID:218482757>).
13. Lamport, Leslie (2004). "Lower Bounds for Asynchronous Consensus" (<http://research.microsoft.com/users/lamport/pubs/pubs.html#lower-bound>).
14. Van Renesse, Robbert; Altınbüken, Deniz (2015-02-17). "Paxos Made Moderately Complex". *ACM Computing Surveys*. **47** (3): 42:1–42:36. doi:10.1145/2673577 (<https://doi.org/10.1145%2F2673577>). ISSN 0360-0300 (<https://search.worldcat.org/issn/0360-0300>).

15. Lamport, Leslie (2005). "Fast Paxos" (<http://research.microsoft.com/users/lamport/pubs/pubs.html#fast-paxos>).
16. Lamport, Leslie (2005). "Generalized Consensus and Paxos" (<http://research.microsoft.com/users/lamport/pubs/pubs.html#generalized>). {{cite journal}}: Cite journal requires |journal= (help)
17. Chandra, Tushar; Griesemer, Robert; Redstone, Joshua (2007). "Paxos made live". *Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing* (<http://research.google.com/pubs/pub33002.html>). pp. 398–407. doi:10.1145/1281100.1281103 (<https://doi.org/10.1145%2F1281100.1281103>). ISBN 978-1-59593-616-5. S2CID 207164635 (<https://api.semanticscholar.org/CorpusID:207164635>).
18. Quesada Torres, Luis (2018). *The Paxos Algorithm* ([https://www.youtube.com/watch?v=d7nAGI\\_NZPk](https://www.youtube.com/watch?v=d7nAGI_NZPk)). Google TechTalks.
19. Lamport, Leslie (2001). Paxos Made Simple (<http://research.microsoft.com/en-us/um/people/lamport/pubs/pubs.html#paxos-simple>) *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001) 51-58.
20. "Leader Election, Why Should I Care?" (<https://www.elastic.co/blog/found-leader-election-in-general>). *Elastic Blog*. 13 September 2013. Retrieved 27 February 2021.
21. I. Gupta, R. van Renesse, and K. P. Birman, 2000, A Probabilistically Correct Leader Election Protocol for Large Groups, *Technical Report*, Cornell University
22. Lamport, Leslie; Massa, Mike (2004). "Cheap Paxos" (<http://research.microsoft.com/users/lamport/pubs/pubs.html#web-dsn-submission>). *Proceedings of the International Conference on Dependable Systems and Networks (DSN 2004)*.
23. Turner, Bryan (2007). "The Paxos Family of Consensus Protocols" (<http://www.fractalscape.org/2007/10/01/paxos-family.html>).
24. Pierre, Sutra; Marc, Shapiro (2011). "Fast Genuine Generalized Consensus" (<http://pagesperso-systeme.lip6.fr/Marc.Shapiro/papers/FGGC-SRDS-2011.pdf>) (PDF). *SRDS'11: 30th IEEE Symposium on Reliable Distributed Systems*.
25. Lamport, Leslie; Malkhi, Dahlia; Zhou, Lidong (2009). "Vertical paxos and primary-backup replication". *Proceedings of the 28th ACM symposium on Principles of distributed computing*. PODC '09. New York, NY, USA: ACM. pp. 312–313. doi:10.1145/1582716.1582783 (<https://doi.org/10.1145%2F1582716.1582783>). ISBN 978-1-60558-396-9. S2CID 2763624 (<https://api.semanticscholar.org/CorpusID:2763624>).
26. Lamport, Leslie; Shostak, Robert; Pease, Marshall (July 1982). "The Byzantine Generals Problem" (<http://research.microsoft.com/users/lamport/pubs/pubs.html#byz>). *ACM Transactions on Programming Languages and Systems*. 4 (3): 382–401. doi:10.1145/357172.357176 (<https://doi.org/10.1145%2F357172.357176>). S2CID 55899582 (<https://api.semanticscholar.org/CorpusID:55899582>). Retrieved 2007-02-02.
27. Castro, Miguel; Liskov, Barbara (February 1999). "Practical Byzantine Fault Tolerance" (<http://pmg.csail.mit.edu/papers/osdi99.pdf>) (PDF). *Proceedings of the Third Symposium on Operating Systems Design and Implementation*: 173–186. Retrieved 5 March 2018.
28. Martin, Jean-Philippe; Alvisi, Lorenzo (July 2006). "Fast Byzantine Consensus" (<https://www.cs.utexas.edu/~lorenzo/papers/fab.pdf>) (PDF). *IEEE Transactions on Dependable and Secure Computing*. 3 (3): 202–215. Bibcode:2006ITDSC...3..202M (<https://ui.adsabs.harvard.edu/abs/2006ITDSC...3..202M>). doi:10.1109/TDSC.2006.35 (<https://doi.org/10.1109%2FTDSC.2006.35>). Retrieved 5 March 2018.
29. Burrows, Mike. "The Chubby lock service for loosely-coupled distributed systems" (<https://static.googleusercontent.com/media/research.google.com/en/archive/chubby-osdi06.pdf>) (PDF). OSDI.
30. "Consensus in the Presence of Partial Synchrony" (<https://groups.csail.mit.edu/tds/papers/Lynch/jacm88.pdf>) (PDF). Archived (<https://web.archive.org/web/20110419165923/http://groups.csail.mit.edu/tds/papers/Lynch/jacm88.pdf>) (PDF) from the original on 2011-04-19. Retrieved 2019-08-31.

31. "Microsoft Research – Emerging Technology, Computer, and Software Research" (<https://www.microsoft.com/en-us/research/?from=https%3A%2F%2Fresearch.microsoft.com%2Fpubs%2F64604%2Fosr2007.pdf&type=no-match>). *Microsoft Research*. Retrieved 2024-09-19.
32. Aahlad et al.(2011). "The Distributed Coordination Engine (DConE)" ([https://www.wandisco.com/get?f=documentation%2Fwhitepapers%2FWANdisco\\_DConE\\_White\\_Paper.pdf](https://www.wandisco.com/get?f=documentation%2Fwhitepapers%2FWANdisco_DConE_White_Paper.pdf)) Archived ([https://web.archive.org/web/20160415093538/https://www.wandisco.com/get?f=documentation%2Fwhitepapers%2FWANdisco\\_DConE\\_White\\_Paper.pdf](https://web.archive.org/web/20160415093538/https://www.wandisco.com/get?f=documentation%2Fwhitepapers%2FWANdisco_DConE_White_Paper.pdf)) 2016-04-15 at the Wayback Machine. WANdisco white paper.
33. Kolbeck, Björn; Höggqvist, Mikael; Stender, Jan; Hupfeld, Felix (2011). "Flease - Lease Coordination without a Lock Server" ([http://www.xtreemfs.org/publications/flease\\_paper\\_ipdps.pdf](http://www.xtreemfs.org/publications/flease_paper_ipdps.pdf)). 25th IEEE International Parallel & Distributed Processing Symposium (IPDPS 2011).
34. "Consistency, Fault Tolerance, and Availability with MariaDB Xpand — MariaDB Documentation" (<https://mariadb.com/docs/xpand/architecture/components/xpand/consistency/>). *MariaDB*. Retrieved 2024-09-19.
35. "Lightweight transactions in Cassandra 2.0" (<https://www.datastax.com/blog/lightweight-transactions-cassandra-20>). *DataStax*. Retrieved 2024-09-19.
36. "Lightweight Transactions | ScyllaDB Docs" (<https://opensource.docs.scylladb.com/stable/using-scylla/lwt.html>). *opensource.docs.scylladb.com*. Retrieved 2024-09-19.
37. Vogels, Dr Werner (2015-07-20). "Under the Hood of Amazon EC2 Container Service" (<http://www.allthingsdistributed.com/2015/07/under-the-hood-of-the-amazon-ec2-container-service.html>). *www.allthingsdistributed.com*. Retrieved 2024-09-19.
38. "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service" (<https://web.archive.org/web/20220719003028/https://www.usenix.org/system/files/atc22-elhemali.pdf>) (PDF). Archived from the original (<https://www.usenix.org/system/files/atc22-elhemali.pdf>) (PDF) on 2022-07-19.